

HCM360 HRIS

Architecture Overview

How the HCM360 HRIS components fit together

Version 1.0.0 · April 2026

1. System Overview

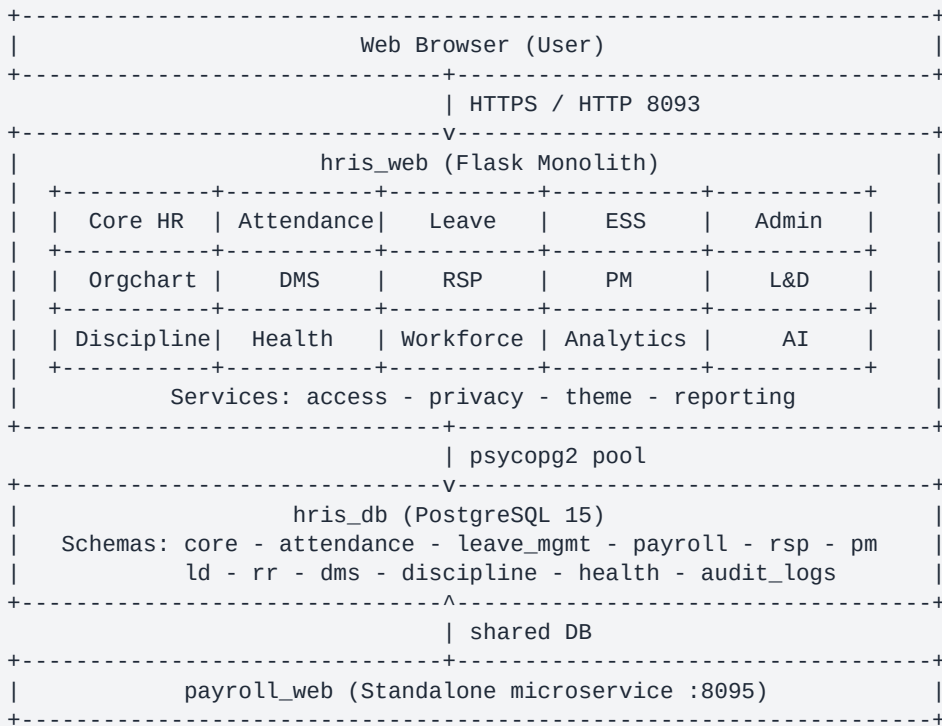
HCM360 is a multi-tenant Human Capital Management system built for Philippine SMEs and enterprises. It is a modular, web-based HRIS with sixteen functional modules spanning core HR, attendance, leave management, payroll, recruitment, performance, learning, rewards, document management, discipline, health and safety, organization charts, workforce planning, analytics, AI assistance, and system administration.

The platform follows a three-tier architecture: a Flask-based web application handling UI and business logic, a PostgreSQL database as the system of record, and a standalone payroll microservice for heavyweight calculations. Containerization via Docker Compose simplifies deployment and keeps all services isolated.

1.1 High-Level Topology

Layer	Technology	Port	Purpose
Presentation	Jinja2 + HTML5 + inline CSS	8093	Server-rendered UI delivered by Flask
Application (Main)	Python 3.11 / Flask	8093 → 5000	Business logic, auth, request routing
Application (Payroll)	Python 3.11 / Flask	8095 → 5001	Payroll calculations & PDF generation
Data	PostgreSQL 15	5440 → 5432	System of record, audit logs, analytics
Storage	Named Docker volume	n/a	File uploads, documents, certificates

1.2 Component Diagram (Logical)



2. Frontend

The frontend uses server-side rendered Jinja2 templates with inline CSS for a zero-build, dependency-free UI. This keeps the deployment footprint small and makes every page cacheable in isolation. A base template provides the navigation shell, theme variables, branding, and session-aware context.

2.1 Template Structure

- templates/base.html — shell, sidebar, theme CSS variables, flash messages
- templates/<module>/*.html — per-module page templates (employees, attendance, leave, payroll, ess_mss, etc.)
- templates/admin/*.html — access matrix, reference data, privacy rules, themes
- Inline CSS using brand color tokens (primary, accent, surface) exposed via g.theme context processor

2.2 Design System Tokens

Token	Default	Usage
--primary	#4F46E5 (indigo)	Buttons, links, highlights, active nav
--accent	#1E40AF (blue)	Brand stripe, cover banners, focus rings
--surface	#FFFFFF	Cards, table rows, modal backgrounds
--text	#111827	Body text, headings
--muted	#6B7280	Labels, captions, placeholder
--border	#E5E7EB	Dividers, card edges, table gridlines

3. Backend Application

The backend is a modular Flask application. Each functional domain lives under ``modules/<name>/'` with its own blueprint, service layer, and templates. Shared concerns (database access, authentication, navigation, theming, reporting, privacy, AI) live under ``services/'`.

3.1 Module Layout

```
hcm360/
├── app.py                # Entry point: app factory, login, before_request
├── config.py             # Config + feature flags from .env
├── requirements.txt       # Python deps
├── Dockerfile            # Web container build
├── docker-compose.yml    # 3-service orchestration
├── services/             # Cross-cutting services
│   ├── db.py            # Connection pool, cursor context manager
│   ├── access_service.py # RBAC: page + feature + mod access
│   ├── privacy_service.py # Field-level RA 10173 masking
│   ├── reference_service.py # 29+ reference tables
│   ├── theme_service.py  # UI theming per company/user
│   ├── reporting_service.py # PDF + Excel generation
│   └── nav_service.py    # Role-filtered navigation tree
├── modules/
│   ├── employees/       # Core HR (master data, 201 file)
│   ├── attendance/      # DTR, shifts, overtime
│   ├── leave_mgmt/      # Leave types, balances, requests
│   ├── payroll/         # Runs, payslips, loans
│   ├── ess_mss/         # Self & manager service
│   ├── orgchart/        # Org hierarchy visualization
│   ├── workforce_planning/ # Headcount scenarios
│   ├── dms/             # 201 documents, certificates
│   ├── rsp/             # Recruitment
│   ├── pm/              # Performance management
│   ├── ld/              # Learning & development
│   ├── rr/              # Rewards & recognition
│   ├── discipline/      # Cases, investigations, sanctions
│   ├── health/          # Incidents, safety reports
│   ├── analytics/       # Dashboards, reports
│   ├── ai/              # AI assistant (ARIA)
│   └── admin/           # System configuration
├── templates/           # Jinja2 templates mirrored by module
├── db/                  # Versioned SQL migrations (00-51)
├── uploads/             # Bind mount for documents
└── payroll/             # Standalone payroll microservice
```

3.2 Request Lifecycle

- 1 Browser issues request to /path. Flask matches blueprint route.
- 2 before_request hook loads g.theme, g.branding, g.users, g.current_user.
- 3 If no user_id in session, redirects to /login (except for public paths).
- 4 can_access_page() checks core.role_page_access and core.user_page_access; returns 403 on miss.
- 5 Blueprint handler calls its service layer (e.g., employees.services.get_employee_detail).
- 6 Service uses get_cursor() context manager from services/db.py to query PostgreSQL.
- 7 Raw data passes through privacy_service.apply_privacy() for field masking.
- 8 Template renders with role-filtered nav, branding, and masked data.
- 9 Response returned with flash messages and session cookie.

4. Data Layer

PostgreSQL 15 hosts all system data. Schemas partition the database by functional domain. Migrations live in db/ and run automatically on first container start via the docker-entrypoint-initdb.d pattern.

4.1 Schema Map

Schema	Purpose	Key Tables
core	Master data, users, access, reference	companies, users, employees, departments, page_registry
attendance	DTR, shifts, biometrics	att_daily, att_shifts, att_overrides
leave_mgmt	Leave types, balances, requests	lv_types, lv_balances, lv_requests, lv_holidays
payroll	Runs, payslips, deductions	pay_runs, pay_employee_payroll, pay_loans, v_payslip
rsp	Recruitment pipeline	candidates, requisitions, interviews
pm	Performance reviews	review_cycles, evaluations, kpis
ld	Training & certifications	trainings, certifications, enrollments
rr	Awards & praise	awards, praise_wall
dms	Documents & certificates	documents, document_categories, certificate_requests
discipline	Cases & sanctions	cases, sanctions, case_types
health	Incidents & safety	incidents, safety_reports
audit_logs	Change tracking	change_log, access_log

5. Integrations

HCM360 is designed to operate as a system of record but can integrate with external systems via standard patterns:

- SSO: JWT token parsing in /login flow (username claim resolution)
- Payroll microservice: shares the same database; communicates via DB state
- AI (ARIA): configurable Anthropic API key via .env for document summaries and query assistance
- Email: optional SMTP configuration for notifications and certificates
- File storage: bind-mounted /uploads volume for persistence across rebuilds

6. Scalability Considerations

- Connection pooling: psycopg2 pool via services/db.py (default max 10 connections)
- Stateless web containers: session in server-signed cookies, easy horizontal scaling
- Read-heavy workloads: materialized views (e.g., payroll.v_payslip) for reporting
- Indexes: every FK + hot query path indexed (employee_id, run_id, date ranges)
- Audit log volume: partitionable by month via pg_partman if needed
- Payroll batch processing: isolated into its own container to avoid blocking the web tier

7. Non-Functional Guarantees

Attribute	Design Decision
Data persistence	Named Docker volumes survive rebuilds; full pg_dump backup in backups/
Privacy compliance	RA 10173 field-level masking with VISIBLE/MASKED/HIDDEN visibility
Audit trail	Triggers on every mutation write to audit_logs.change_log
Auth	Session-based with role-driven page/feature/mod access matrix
Restartability	restart: unless-stopped for all containers
Upgradability	Versioned SQL migrations; each change adds a new numbered file